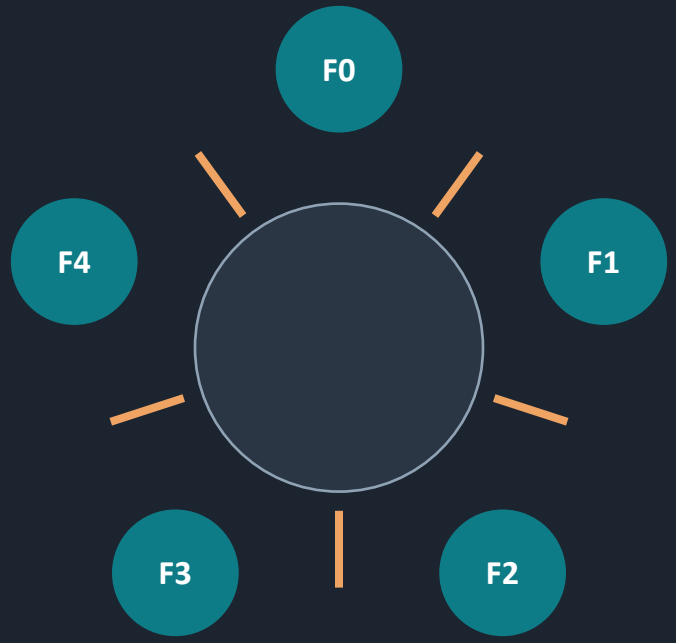
KARADENİZ TEKNİK ÜNİVERSİTESİ

Yazılım Geliştirme Bölümü

İŞLETİM SİSTEMLERİ

Hafta 7

Senkronizasyon II: Klasik Problemler, Monitörler ve Gerçek Sistemler



Dr. Öğr. Üyesi Sayyad Alizadeh

Bu Haftanın Yol Haritası

1

Sınırlı tampon

Üç semafor: mutex, empty, full

2

Yanlış sıra

wait() sırası neden kilitlenmeye yol açabilir?

3

Okuyucular–yazarlar

Aynı veriye iki farklı erişim türü

4

Yemek yiyen filozoflar

Birden çok kaynağı birlikte almak

5

Monitörler

Kapsüllenmiş karşılıklı dışlama

6

Koşul değişkenleri

wait(), signal() ve semantik farklar

7

Gerçek sistemler

Pthreads, Windows, Linux

8

Alternatif yaklaşımlar

İşlemsel bellek, OpenMP, bariyer, RCU

Öğrenme Çıktıları

Bu haftanın sonunda öğrenciler:

1. Sınırlı tampon problemini üç semaforla çözer ve semafor değerlerini adım adım izler
2. Yanlış wait() sırasının neden kilitlenmeye yol açabileceğini gösterir
3. Birinci okuyucular–yazarlar çözümünü izler; yazar açlığını açıklar
4. Yemek yiyen filozofların naif çözümünde kilitlenmeye götüren sırayı bulur
5. Monitör ve koşul değişkeni semantiğini (wait, signal, while) açıklar
6. Pthreads, Windows ve Linux araçlarını ve alternatif yaklaşımları sınıflandırır

Geçen Haftadan Bu Haftaya

HAFTA 6'NIN ARAÇLARI

Yarış durumu, kritik bölge	**Mutex ve döner kilit**
Semafor: wait() / signal()	**Kural: önce azalt; < 0 ise bekle**

- Hafta 6 araçları yeniden anlatılmaz
- Bu hafta onları gerçek problemleri çözmek için kullanıyoruz
- Semafor kuralı Hafta 6 ile aynıdır: değer negatif olabilir; |değer| = bekleyen sayısı

BU HAFTANIN TEMEL SORUSU

Mutex ve semaforları biliyoruz. Gerçek eşzamanlılık problemlerini bunlarla nasıl çözeriz?

Ve sonra: hata yapmayı zorlaştıran daha üst düzey bir yapı var mı? → monitörler

Senkronizasyonun Üç Klasik Problemi

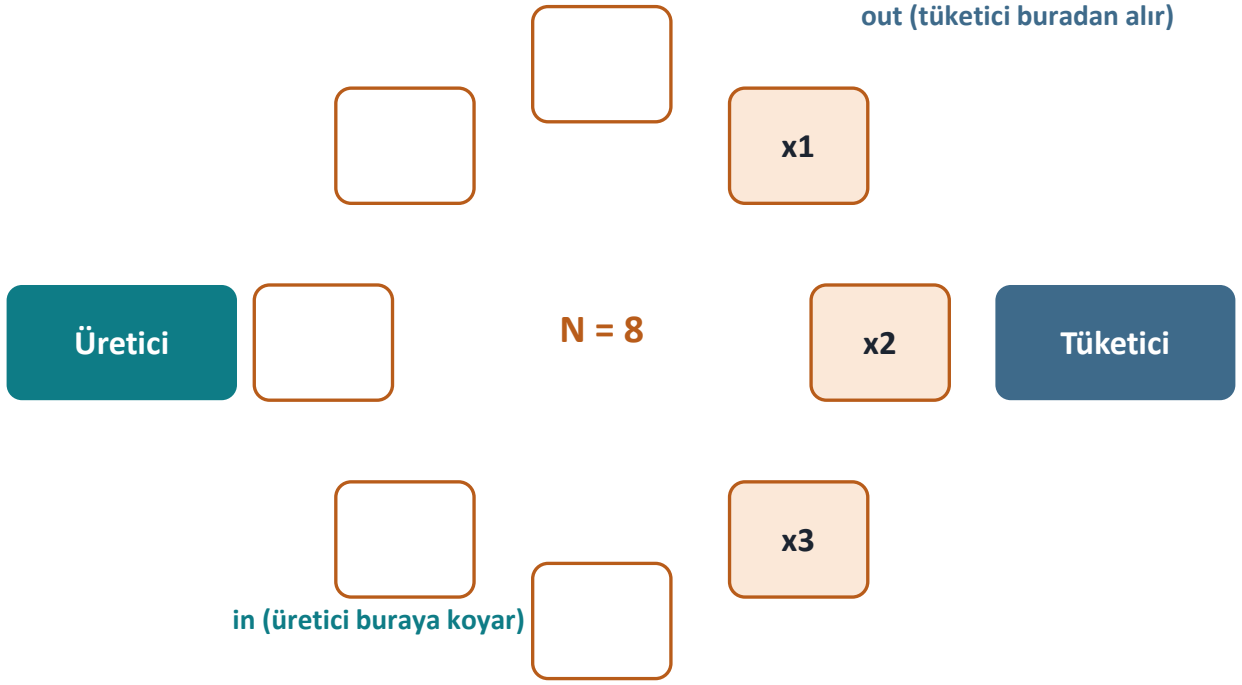
Sınırlı tampon Üretici–tüketici	Okuyucular–yazarlar Ortak veritabanı	Yemek yiyen filozoflar Beş filozof, beş çatal
Temsil ettiği: Kaynak sayma + karşılıklı dışlama	**Temsil ettiği:** Farklı erişim türleri	**Temsil ettiği:** Birden çok kaynağı birlikte almak
Zorluk: Tampon dolu / boş iken bekleme	**Zorluk:** Okuyucular paylaşabilir, yazar tek başına	**Zorluk:** Kilitlenme ve açlık riski

Bunlar yalnızca akademik oyun değildir: yeni senkronizasyon yöntemleri bu problemlerle sınanır; gerçek sistemlerdeki kuyruklar, veritabanları ve kaynak paylaşımı bu kalıplara benzer.

Kaynaklar: Silberschatz vd., Bölüm 5.7; Tanenbaum & Bos, Bölüm 2.5

Sınırlı Tampon Problemi

- N gözlü ortak tampon
- Üretici tampona öğe koyar; tüketici öğe alır
- Tampon DOLUYKEN üretici beklemeli
- Tampon BOŞKEN tüketici beklemeli
- Tamponun kendisine (in, out, gözler) eşzamanlı erişim korunmalı



Bu hafta artık counter yok: semaforlar sayar.

Üç Semafor ve Başlangıç Değerleri

`mutex = 1`	`empty = N`	`full = 0`
Tampon veri yapısını (gözler, in, out) korur	**Boş göz sayısını sayar**	**Dolu göz sayısını sayar**
Neden? 1: başlangıçta tampona kimse erişmiyor	**Neden?** N: başlangıçta tüm gözler boş	**Neden?** 0: başlangıçta tüketilecek öğe yok

Rol ayrımı: mutex → karşılıklı dışlama · empty ve full → kaynak sayma ve bekleme. empty ve full karşılıklı dışlama sağlamaz!

Üretici: Her wait() ve signal() Neden Orada?

```
do {
    /* bir öğe üret */
    wait(empty);     /* boş göz? */
    wait(mutex);     /* tamponu kilitle */
    /* öğeyi tampona ekle */
    signal(mutex);   /* kilidi bırak */
    signal(full);    /* dolu göz +1 */
} while (true);
```

wait(empty) — Tampon doluysa (empty ≤ 0) burada bekler; boş göz ayırtır

wait(mutex) — Gözlere ve in değişkenine tek başına erişim

signal(mutex) — Tamponu en kısa sürede bırakır

signal(full) — Bekleyen bir tüketici varsa uyandırır

Önce SAYMA semaforu (empty), sonra KİLİT (mutex): beklenecekse kilit tutulmadan beklenir. Çıkışta kilit önce bırakılır.

Silberschatz vd., Şekil 5.9

Üretici ve Tüketici Yan Yana

Üretici

```
wait(empty);
wait(mutex);
  /* öğe ekle */
signal(mutex);
signal(full);
```

Tüketici

```
wait(full);
wait(mutex);
  /* öğe çıkar */
signal(mutex);
signal(empty);
```

Konum	Üretici	Tüketici	Simetri
İlk wait	`empty`	`full`	Her biri kendi “sayılan kaynağını” bekler
İkinci wait	`mutex`	`mutex`	Aynı kilit
Son signal	`full`	`empty`	Her biri karşı tarafın kaynağını artırır

Kaynak: Silberschatz vd., Bölüm 5.7.1 (Şekil 5.9–5.10)

Çözümlü İzleme: N = 3

Başlangıç: mutex = 1, empty = 3, full = 0 · Her satır bir üret/tüket işleminin tamamıdır (mutex satır sonunda yine 1).

Adım	Olay	mutex	empty	full	Dolu göz	Bekleyen	Açıklama
0	Başlangıç	1	3	0	0	–	tampon boş
1	**Üretici üretir**	1	2	1	1	–	empty−1, full+1
2	**Üretici üretir**	1	1	2	2	–	
3	**Tüketici tüketir**	1	2	1	1	–	full−1, empty+1
4	**Üretici üretir**	1	1	2	2	–	
5	**Üretici üretir**	1	0	3	3	–	tampon DOLU
6	**Üretici wait(empty)**	1	−1	3	3	Üretici	empty < 0 → bekler
7	**Tüketici tüketir**	1	0	2	2	–	signal(empty): −1 → 0, üreticiyi uyandırır
8	**Uyanan üretici ekler**	1	0	3	3	–	wait(empty) zaten geçilmişti

Denetim: kimse beklemiyorken empty + full = 3 ve dolu göz = full. Hafta 6 kuralı: önce azalt; < 0 ise bekle; signal'da ≤ 0 ise uyandır.

Yanlış Sıra: Üreticide wait(mutex) Önce Olursa

```
/* YANLIŞ üretici */
wait(mutex);
wait(empty);
  /* öğe ekle */
signal(mutex);
signal(full);
```

Tampon dolu değilse aynı kod çalışıyor gibi görünür: hata yalnızca belirli bir durumda ortaya çıkar.

Başlangıç: tampon DOLU (N = 3) → mutex = 1, empty = 0, full = 3

Adım	Olay	mutex	empty	full	Durum
1	Üretici: wait(mutex)	0	0	3	üretici kilidi aldı
2	Üretici: wait(empty)	0	−1	3	**üretici bekler — kilidi TUTARAK**
3	Tüketici: wait(full)	0	−1	2	geçer
4	Tüketici: wait(mutex)	−1	−1	2	**tüketici bekler (kilit üreticide)**

Üretici, yalnızca tüketicinin yapabileceği signal(empty)'yi bekliyor. Tüketici, üreticinin tuttuğu mutex'i bekliyor. → İkisi de sonsuza dek bekler: kilitlenme (deadlock) oluşabilir.

Tüketicide wait(mutex) önce olursa simetrik olarak tampon BOŞKEN aynı sorun yaşanır.

Çıkarılacak İlke

Beklemek zorunda kalabileceğin bir kaynağı beklerken, başka bir sürecin ilerlemek için ihtiyaç duyduğu kilidi gereksiz yere tutma.

Üretici, mutex'i tutarken empty bekler
tehlikeli durum: tampon dolu

Tüketici, mutex'i tutarken full bekler
tehlikeli durum: tampon boş

Ortak yapı: bir süreç bir kaynağı TUTUYOR ve başka bir kaynağı BEKLİYOR; diğer süreç de tersini yapıyor.

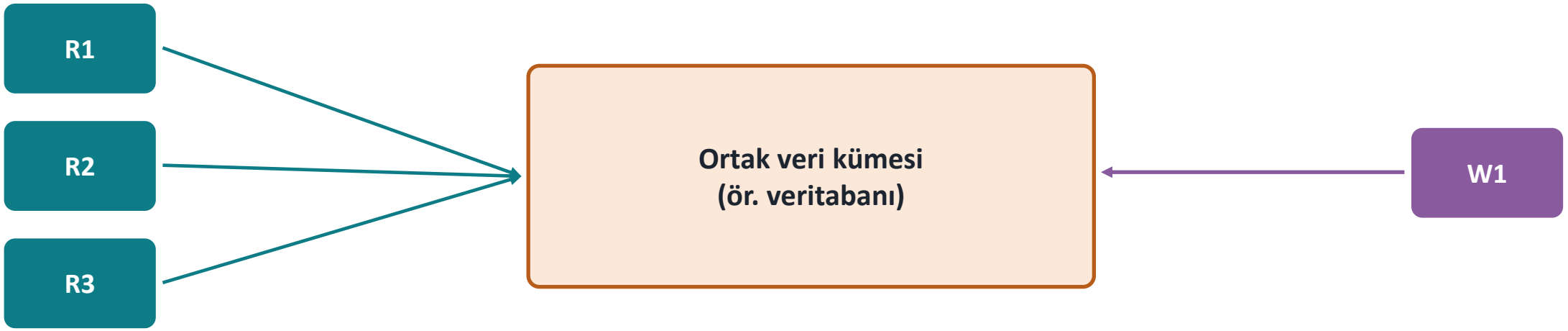
Bu yapının sistematik analizi (koşullar, kaynak atama grafiği, önleme) → Hafta 8: Kilitlenmeler

Sınırlı Tampon: Hangi Problem, Hangi Araç?

Problem	Araç	Başlangıç
Tampon yapısına aynı anda erişim	`mutex`	1
Boş göz sayısı	`empty`	N
Dolu göz sayısı	`full`	0
Üretici tampon doluyken beklemeli	`wait(empty)`	–
Tüketici tampon boşken beklemeli	`wait(full)`	–
Üretici sonrası tüketiciye haber	`signal(full)`	–
Tüketici sonrası üreticiye haber	`signal(empty)`	–

Kaynak: Silberschatz vd., Bölüm 5.7.1

Okuyucular–Yazarlar Problemi



Okuyucular: yalnızca okur

Yazar: okur ve DEĞİŞTİRİR

İzin verilen

Birden çok okuyucu aynı anda okuyabilir: veriyi değiştirmedikleri için birbirlerini bozmazlar.

İzin verilmeyen

Yazar yazarken başka hiçbir yazar ya da okuyucu veriye erişemez: yazar ayrıcalıklı erişim ister.

Kaynaklar: Silberschatz vd., Bölüm 5.7.2; Tanenbaum & Bos, Bölüm 2.5.2

Hangi Erişimler Aynı Anda Olabilir?

İçeride olan ↓ / gelen →	Okuyucu gelir	Yazar gelir
Kimse yok	girer	girer
Bir ya da daha çok okuyucu	girer (paylaşımlı okuma)	bekler
Bir yazar	bekler	bekler

Özetle: okuyucu + okuyucu → izin var · okuyucu + yazar → yok · yazar + yazar → yok

Birinci Okuyucular–Yazarlar Çözümü: Değişkenler

Amaç (okuyucu önceliği): Yazar zaten izin almadıysa, hiçbir okuyucu sırf bir yazar bekliyor diye bekletilmez.

```
int read_count = 0;
```

Şu anda okuyan okuyucu sayısı

```
semaphore mutex = 1;
```

read_count'u korur (yalnızca okuyucular kullanır)

```
semaphore rw_mutex = 1;
```

Ortak veriyi yazara karşı korur (yazarlar ve ilk/son okuyucu)

Kaynak: Silberschatz vd., Bölüm 5.7.2

Okuyucu Kodu: İlk Okuyucu Kilitler, Son Okuyucu Açar

```
wait(mutex);
read_count++;
if (read_count == 1)
    wait(rw_mutex);      /* İLK okuyucu */
signal(mutex);

/* OKUMA */

wait(mutex);
read_count--;
if (read_count == 0)
    signal(rw_mutex);    /* SON okuyucu */
signal(mutex);
```

Silberschatz vd., Şekil 5.12

Giriş: read_count'u güvenle artır; ilk okuyucuysan yazarları dışarıda tut

İlk okuyucu: rw_mutex'i alır: artık yazar giremez; sonraki okuyucular rw_mutex'i beklemez

Okuma: Birden çok okuyucu aynı anda burada olabilir

Son okuyucu: rw_mutex'i bırakır: bekleyen yazar varsa girebilir

Neden read_count İçin Ayrı Bir mutex Var?

read_count da paylaşılan bir değişkendir: iki okuyucu aynı anda read_count++ yaparsa Hafta 6'daki yarış durumu oluşur.

`mutex`

korur: read_count

Yalnızca okuyucular kullanır · çok kısa süre tutulur (sayaç güncellemesi)

`rw_mutex`

korur: ortak veri kümesi

Yazarlar ve okuyucu GRUBU arasında · grup adına ilk okuyucu alır, son okuyucu bırakır

İki farklı koruma ihtiyacı → iki farklı semafor. Tek semaforla okuyucuların paylaşımlı okuması mümkün olmazdı.

Yazar Kodu: Neden Bu Kadar Basit?

```
wait(rw_mutex);

/* YAZMA */

signal(rw_mutex);
```

- Yazar her zaman TEK başına erişmek ister: sayma gerekmez
- rw_mutex alındıysa ya bir yazar ya da okuyucu grubu içeridedir
- Okuyucular gruptur: grubun ilk girişini ve son çıkışını izlemek için read_count gerekir

Ölçüt	Okuyucu	Yazar
Kullandığı semaforlar	mutex, rw_mutex (yalnızca ilk/son)	rw_mutex
Eşzamanlı erişim	Diğer okuyucularla birlikte	Tek başına
Sayaç	read_count	yok

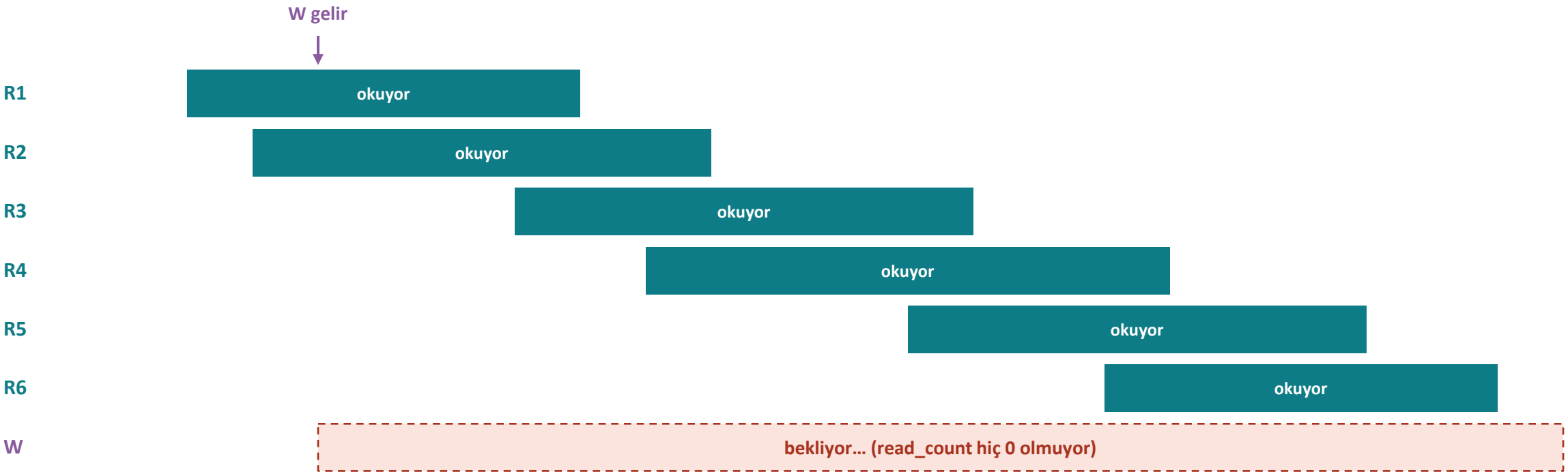
Silberschatz vd., Şekil 5.11

Çözümlü İzleme: Okuyucular ve Bir Yazar

Adım	Olay	read_count	mutex	rw_mutex	Okuyanlar	Yazar	Açıklama
0	Başlangıç	0	1	1	–	–	
1	**R1 gelir**	1	1	0	R1	–	ilk okuyucu rw_mutex'i alır
2	**R2 gelir**	2	1	0	R1, R2	–	rw_mutex'i beklemez
3	**W1 gelir**	2	1	−1	R1, R2	bekliyor	wait(rw_mutex) → bekler
4	**R3 gelir**	3	1	−1	R1, R2, R3	bekliyor	okuyucu, bekleyen yazarı geçer
5	**R1 çıkar**	2	1	−1	R2, R3	bekliyor	
6	**R2 çıkar**	1	1	−1	R3	bekliyor	
7	**R3 çıkar**	0	1	0	–	uyanır	son okuyucu signal(rw_mutex)
8	**W1 yazar**	0	1	0	–	YAZIYOR	tek başına
9	**W1 bitirir**	0	1	1	–	–	signal(rw_mutex)

mutex her satırın sonunda yine 1'dir (read_count güncellemesi bitmiştir). Okuyucu ile yazar hiçbir satırda aynı anda veriye erişmez.

Birinci Çözümün Sorunu: Yazar Açlığı



Ne oluyor?

Yeni okuyucular eskiler çıkmadan geliyor; okuyucu grubu hiç boşalmıyor ve yazar sonsuza dek bekleyebiliyor.

Hafta 6 bağlantısı

Bu bir açlıktır (starvation), kilitlenme değildir: okuyucular ilerlemeye devam eder.

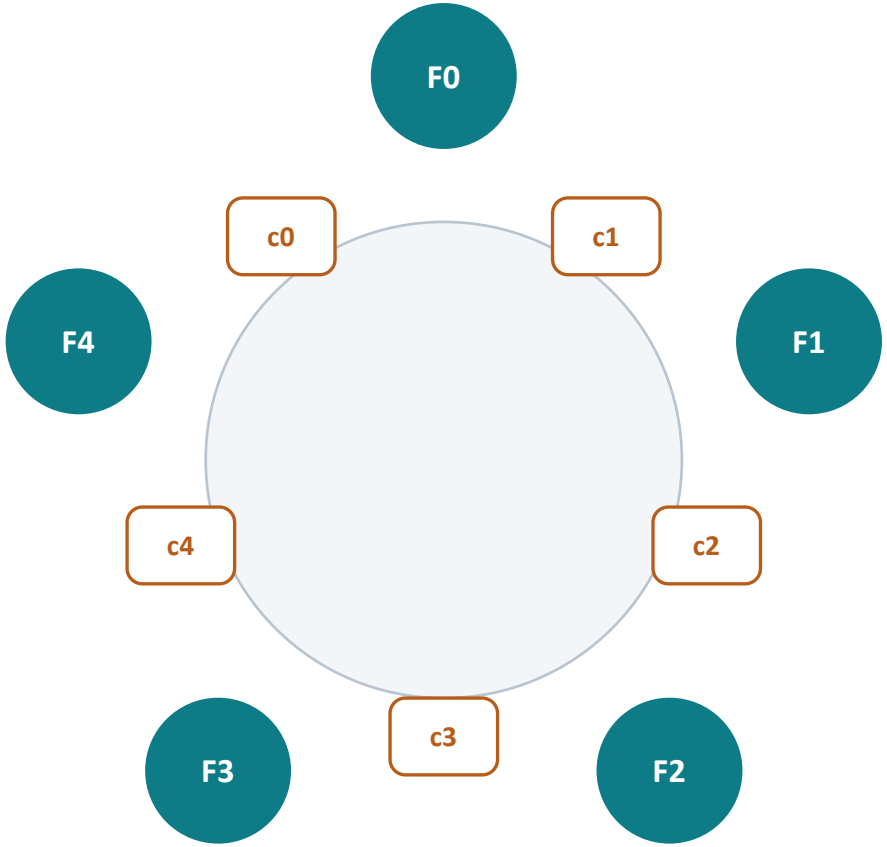
Üç Yaklaşım: Okuyucu Önceliği, Yazar Önceliği, Adil

Yaklaşım	Kural	Avantaj	Risk
Okuyucu önceliği (birinci problem)	Yazar içeride değilse okuyucular hiç beklemez	Yüksek okuma eşzamanlılığı	Yazar açlığı
Yazar önceliği (ikinci problem)	Bir yazar hazırsa yeni okuyucular onun önüne geçemez	Güncellemeler gecikmez	Okuyucu açlığı
Adil (açlıksız)	Geliş sırası ya da adil bir kuyruk korunur	Kimse sonsuza dek beklemez	Daha az eşzamanlılık, daha karmaşık

Gerçek sistemlerde okuyucu–yazar kilitleri (reader–writer locks) bulunur; özellikle okumanın yazmadan çok daha sık olduğu durumlarda yararlıdır.

İkinci ve adil çözümlerin kodu bu derste verilmemiştir. Kaynak: Silberschatz vd., Bölüm 5.7.2

Yemek Yiyen Filozoflar Problemi



- Beş filozof yuvarlak masada; aralarında beş çubuk (çatal)
- Filozof ya düşünür ya acıkır ya da yer: DÜŞÜNÜYOR → AÇ → YİYOR
- Yemek için HEM sol HEM sağ çubuğu almalıdır; tek seferde bir çubuk alabilir
- Komşular aynı çubuğu paylaşır: komşu iki filozof aynı anda yiyemez

Temsil ettiği: birden çok kaynağı kilitlenme ve açlık olmadan birlikte ayırma.

c_i: filozof F_i'nin SOLUNDAKİ çubuk · F_i'nin sağındaki çubuk c_(i+1) mod 5

Kaynaklar: Silberschatz vd., Bölüm 5.7.3; Tanenbaum & Bos, Bölüm 2.5.1

Naif Semafor Çözümü

```
semaphore chopstick[5];   /* hepsi 1 */

/* Filozof i */
do {
    wait(chopstick[i]);              /* sol */
    wait(chopstick[(i + 1) % 5]);    /* sağ */
    /* YE */
    signal(chopstick[i]);
    signal(chopstick[(i + 1) % 5]);
    /* DÜŞÜN */
} while (true);
```

Silberschatz vd., Şekil 5.14

İlk bakışta doğru

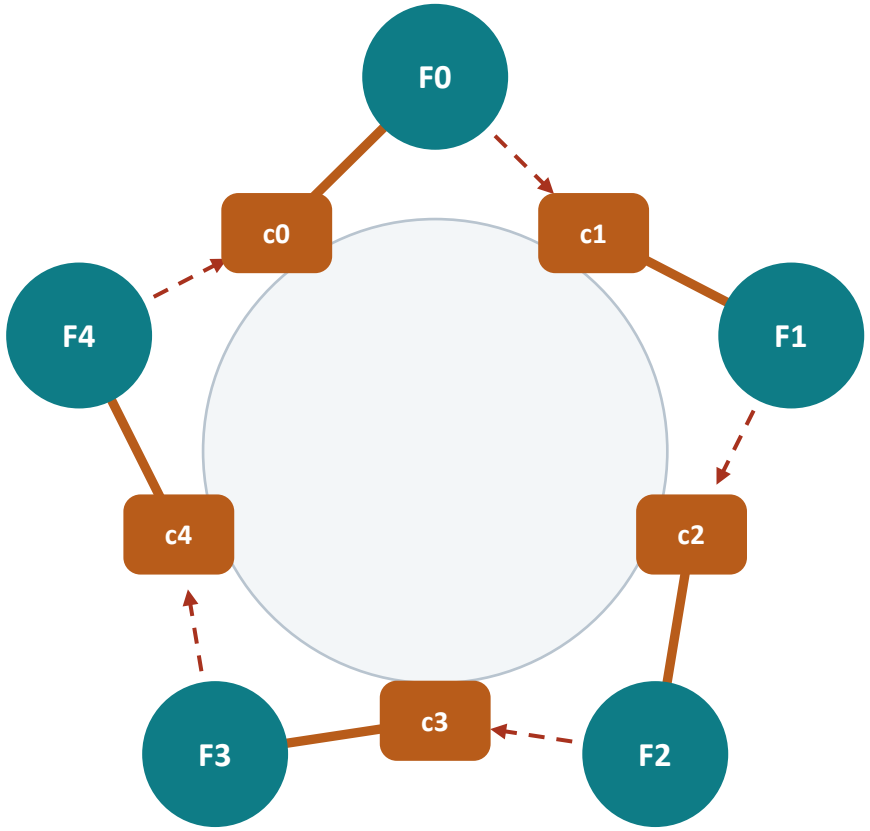
- Her çubuk bir semafor: aynı anda tek filozof tutar
- Komşu iki filozof aynı anda yiyemez

Soru: Beş filozof aynı anda acıkırsa ne olur?

Kilitlenmeye Götüren Yürütme

Adım	Olay	c0	c1	c2	c3	c4
0	Başlangıç	1	1	1	1	1
1	F0: wait(c0)	0	1	1	1	1
2	F1: wait(c1)	0	0	1	1	1
3	F2: wait(c2)	0	0	0	1	1
4	F3: wait(c3)	0	0	0	0	1
5	F4: wait(c4)	0	0	0	0	0
6	F0: wait(c1) → bekler	0	−1	0	0	0
7	F1: wait(c2) → bekler	0	−1	−1	0	0
8	F2: wait(c3) → bekler	0	−1	−1	−1	0
9	F3: wait(c4) → bekler	0	−1	−1	−1	−1
10	F4: wait(c0) → bekler	−1	−1	−1	−1	−1

F0 → F1 → F2 → F3 → F4 → F0: herkes sağındakinin tuttuğu çubuğu bekliyor. Kimse ilerleyemez.



Turuncu çizgi: tutulan çubuk · kesikli kırmızı ok: beklenen çubuk

Çözüm Fikirlerine Kısa Bakış

1

En fazla dört filozof

Aynı anda en fazla dört filozofun masaya oturmasına izin ver: en az birinin iki çubuğu olur

2

İki çubuk birlikte

Filozof çubukları yalnızca ikisi de boşsa alsın (bu kontrol kritik bölgede yapılmalı)

3

Asimetrik sıra

Tek numaralı filozof önce sol sonra sağ, çift numaralı önce sağ sonra sol çubuğu alsın

Kilitlenmeyi önleyen bir çözüm açlığı her zaman önlemez: her filozofun sonunda yiyebileceği ayrıca gösterilmelidir.

Neden kilitlenme oluşur ve nasıl sistematik önlenir? → Hafta 8

Kaynak: Silberschatz vd., Bölüm 5.7.3

Semafor Çözümlerinin Ortak Sorunu

Semafor güçlü ve esnektir; ama wait() / signal() yerleştirmek tamamen programcının sorumluluğundadır.

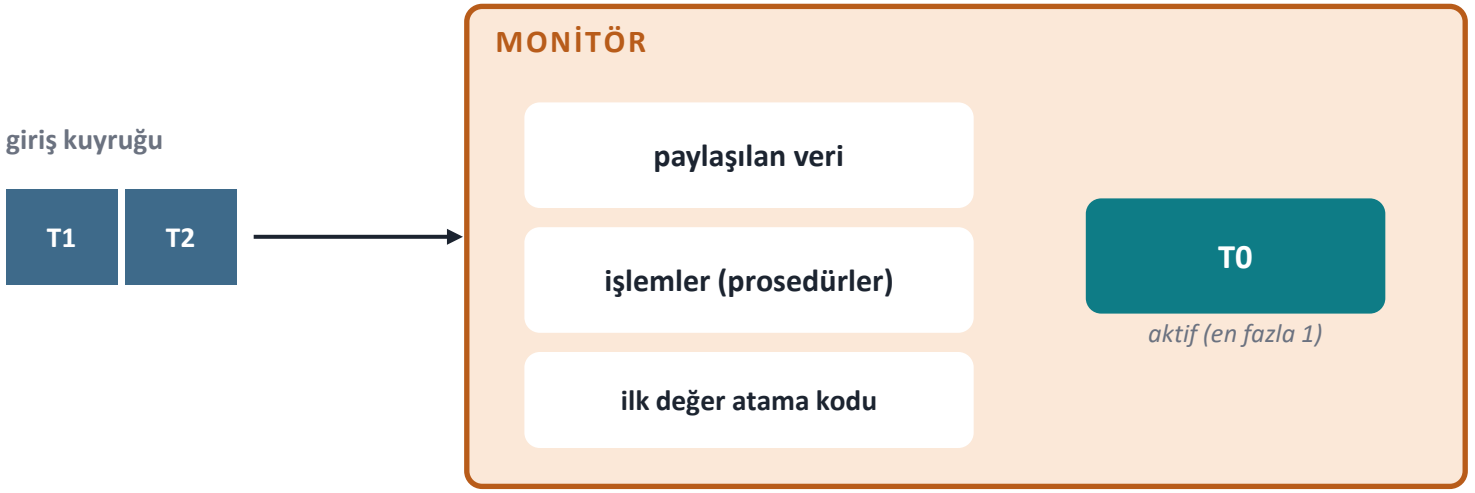
Sıra yanlış	signal ile wait karıştı	Unutulan çağrı	Politika
`Sınırlı tamponda wait(mutex) önce`	`signal(mutex) … wait(mutex)`	`release / signal yok`	`okuyucu önceliği`
↓	↓	↓	↓
kilitlenme	**karşılıklı dışlama bozulur**	**diğerleri sonsuza dek bekler**	**açlık**

Hatalar zamanlamaya bağlıdır: her çalıştırmada görülmez. Daha üst düzeyli, hata yapmayı zorlaştıran bir yapı olabilir mi? → MONİTÖR

Kaynak: Silberschatz vd., Bölüm 5.8

Monitör (Monitor) Nedir?

Monitör: Paylaşılan veriyi, bu veriye erişen işlemleri ve senkronizasyonu bir arada kapsülleyen üst düzey senkronizasyon yapısıdır (soyut veri türü).



- Paylaşılan veriye YALNIZCA monitörün işlemleri erişebilir
- Monitör içinde aynı anda yalnızca BİR süreç / iş parçacığı aktif olabilir
- Bu karşılıklı dışlamayı programcı değil, monitör yapısı sağlar

Kaynaklar: Silberschatz vd., Bölüm 5.8.1 (Şekil 5.16); Tanenbaum & Bos, Bölüm 2.3.7

Monitörün Söz Dizimi (Kavramsal)

```
monitor ornek {
    /* paylaşılan değişkenler */

    procedure P1(...) {
        ...
    }
    procedure P2(...) {
        ...
    }

    initialization code(...) {
        ...
    }
}
```

Ne olur?

T0 ornek.P1() çağırır
monitöre girer, aktif olur

T1 ornek.P2() çağırır
monitör dolu → giriş kuyruğunda bekler

T0 P1'den döner
monitör boşalır

T1 girer
artık aktif olan T1

Monitör bir programlama dili yapısıdır (ör. Concurrent Pascal, Java'nın synchronized yapısı benzer fikri uygular).

Monitör Neden Daha Güvenli? Ve Neden Yetmez?

Semaforla

- Ortak verinin her erişimine programcı wait/signal yerleştirir
- Bir yerde unutmak ya da sırayı karıştırmak hataya yol açar

Monitörle

- Veriye yalnızca monitör işlemleriyle erişilir
- Karşılıklı dışlama otomatik: derleyici / çalışma zamanı sağlar

Ama yetmez: Karşılıklı dışlama tek başına yeterli değildir. Ör. tüketici monitör içindeyken tampon boşsa ne yapmalı? Bir KOŞUL beklemesi gerekir.

Çözüm: koşul değişkenleri (condition variables)

Koşul Değişkeni (Condition Variable)

```
condition x, y;

x.wait();      /* çağıran askıya alınır */
x.signal();    /* bekleyen birini sürdür */
```

- x.wait(): çağıran x'in kuyruğunda bekler VE monitörü başkasına bırakır
- x.signal(): x'te bekleyen varsa tam olarak birini sürdürür
- Bekleyen yoksa signal'ın HİÇBİR etkisi yoktur (semafordaki signal'dan farkı: durum saklanmaz)



Koşul kuyruklarındaki iş parçacıkları monitörün içinde sayılmaz: beklerken aktif değildir. Silberschatz vd., Bölüm 5.8.1 (Şekil 5.17)

Neden wait() Monitörü Bırakmak Zorunda?

Senaryo: T0 (tüketici) monitörde; tampon boş. T1 (üretici) öğe eklemek istiyor.

Monitörü TUTARAK beklerse (yanlış)

1. T0 bekler; monitör hâlâ T0'da
2. T1 monitöre giremez
3. Tampon hiç dolmaz

T0 sonsuza dek bekler

wait() monitörü BIRAKIR (doğru)

1. T0: notEmpty.wait() → bekler + monitörü bırakır
2. T1 girer, öğe ekler, notEmpty.signal()
3. T0 sürdürülür, öğeyi alır

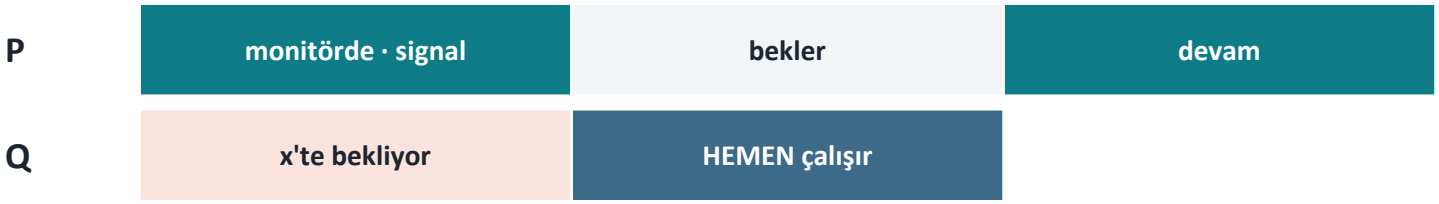
Herkes ilerler

wait() iki işi birlikte (atomik olarak) yapar: koşul kuyruğunda beklemeye geç + monitörü bırak.

signal() Sonrası Kim Çalışır? İki Semantik

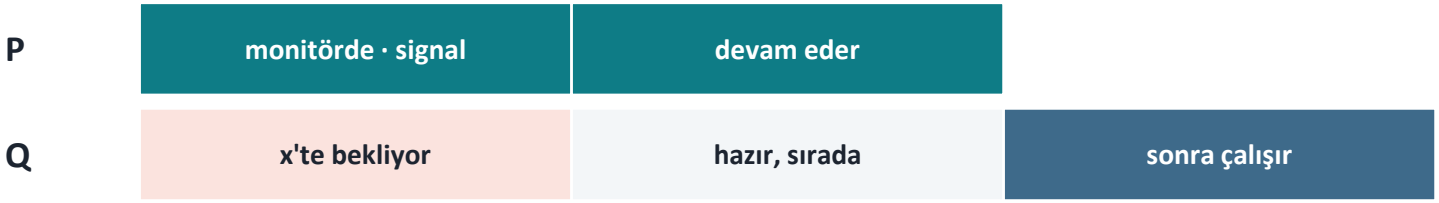
P, x.signal() çağırır; Q, x'te bekliyordu. Monitörde aynı anda yalnızca biri aktif olabilir:

Signal-and-Wait (Hoare tarzı)



Uyandırılan Q hemen çalışır; P, Q çıkana ya da başka koşulda bekleyene kadar bekler.

Signal-and-Continue (Mesa tarzı)



P devam eder; Q hazır olur ve P çıktıktan sonra (ya da başka bir koşulda beklediğinde) çalışır.

İkisi de mantıklıdır; farklı monitör gerçekleştirmeleri farklı semantik seçer (ör. Concurrent Pascal ara yol: signal yapan hemen monitörden çıkar). Pthreads koşul değişkenleri signal-and-continue gibi davranır.

Monitörle Sınırlı Tampon

```
monitor BoundedBuffer {
    item buffer[N];  int count = 0;
    condition notFull, notEmpty;

    void insert(item x) {
        while (count == N) notFull.wait();
        /* x'i ekle */  count++;
        notEmpty.signal();
    }
    item remove() {
        while (count == 0) notEmpty.wait();
        /* öğeyi çıkar */  count--;
        notFull.signal();
    }
}
```

Karşılıklı dışlama: Monitör sağlar: mutex semaforu yok

notFull: tampon doluyken üretici burada bekler (empty'nin rolü)

notEmpty: tampon boşken tüketici burada bekler (full'un rolü)

count: tampondaki öğe sayısı — monitör içinde güvenle güncellenir

Monitör sözde kodu (Hoare'un sınırlı tampon örneğine benzer); while kullanımı bir sonraki slaytta açıklanır.

Önemli: if mi, while mı?

if (riskli)

```
if (count == 0)
    notEmpty.wait();
```

Uyanınca koşulun hâlâ doğru olduğu varsayılır. Signal-and-continue'da sinyal ile çalışma arasında başka bir tüketici öğeyi almış olabilir → boş tampondan okuma!

while (güvenli)

```
while (count == 0)
    notEmpty.wait();
```

Uyanınca koşul YENİDEN sınanır; hâlâ sağlanmıyorsa tekrar beklenir. Gerçek kullanımlarda (ör. Pthreads) önerilen biçim budur.

“signal geldi → koşul kesinlikle doğru” düşüncesi yanlıştır. Sinyal yalnızca “koşul değişmiş olabilir, kontrol et” anlamına gelir.

Hoare semantiğinde uyanan hemen çalıştığı için if yeterli olabilir; ama while her iki semantikte de doğrudur.

Çözümlü İzleme: Monitör ve Koşul Değişkeni

BoundedBuffer monitörü (N = 2), signal-and-continue semantiği. Başlangıç: count = 0.

Adım	Olay	Monitörde aktif	Giriş kuyruğu	notEmpty kuyruğu	count
1	C: remove() çağırır	C	–	–	0
2	C: count == 0 → notEmpty.wait()	–	–	C	0
3	P: insert() çağırır, girer	P	–	C	0
4	P: öğe ekler	P	–	C	1
5	P: notEmpty.signal()	P	C (hazır)	–	1
6	P: insert()'ten döner	–	C	–	1
7	C: monitörü yeniden alır; while(count==0)? hayır	C	–	–	1
8	C: öğeyi çıkarır; notFull.signal() (bekleyen yok)	C	–	–	0
9	C: remove()'dan döner	–	–	–	0

Her satırda monitörde en fazla bir aktif iş parçacığı vardır. 8. adımda bekleyen olmadığı için signal'ın etkisi yoktur.

Monitörle Yemek Yiyen Filozoflar

```
monitor DiningPhilosophers {
  enum {THINKING, HUNGRY, EATING} state[5];
  condition self[5];

  void pickup(int i) {
    state[i] = HUNGRY;  test(i);
    if (state[i] != EATING) self[i].wait();
  }
  void putdown(int i) {
    state[i] = THINKING;
    test((i + 4) % 5);  test((i + 1) % 5);
  }
  void test(int i) {
    if (state[(i+4)%5] != EATING && state[i] == HUNGRY
        && state[(i+1)%5] != EATING) {
      state[i] = EATING;  self[i].signal();
    }
  }
}
```

`pickup(i)`
acıkır; iki komşu yemiyorsa yer, yoksa self[i]'de bekler

`test(i)`
i aç VE iki komşusu yemiyorsa i'yi YİYOR yapar ve uyandırır

`putdown(i)`
düşünmeye döner; sol ve sağ komşunun artık yiyip yiyemeyeceğini sınar

Filozof çubukları ancak İKİSİ de boşken (iki komşu yemiyorken) alır.

Kullanım: DiningPhilosophers.pickup(i); … ye … DiningPhilosophers.putdown(i); Silberschatz vd., Şekil 5.18

Monitör Çözümü Neyi Garanti Eder, Neyi Etmez?

Garanti eder

- İki komşu filozof aynı anda yemez
- Kilitlenme oluşmaz: bir filozof tek çubukla bekleyip diğerini tutmaz; ya ikisini birden alır ya hiçbirini

Garanti etmez

- Açlık hâlâ mümkündür: iki komşusu sırayla yemeye devam ederse bir filozof sürekli bekleyebilir
- Programcının pickup ve putdown'ı doğru sırayla çağırması gerekir

Monitör hata yapmayı zorlaştırır ama tüm senkronizasyon problemlerini otomatik çözmez: kilitlenme ve açlık ayrıca değerlendirilmelidir.

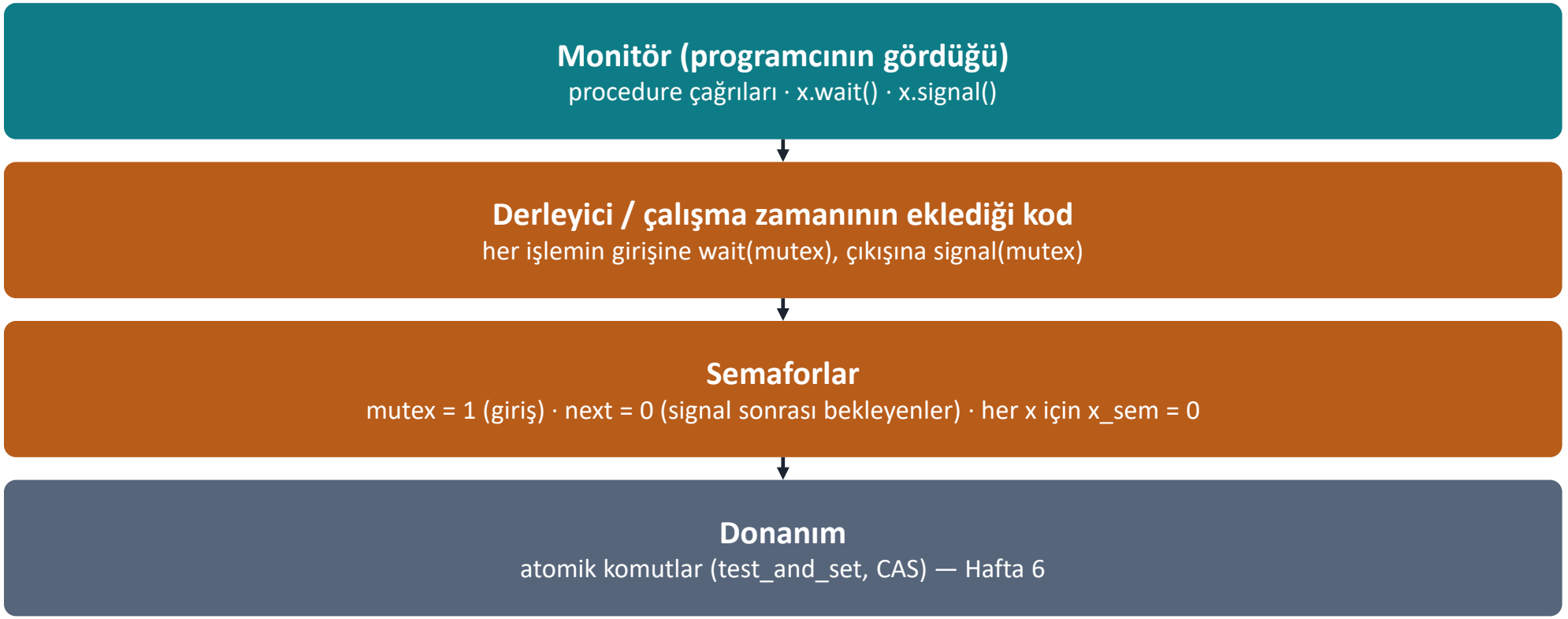
Kaynak: Silberschatz vd., Bölüm 5.8.2

Semafor ile Monitör Karşılaştırması

Özellik	Semafor	Monitör
Soyutlama düzeyi	Düşük: bir tamsayı + iki işlem	Yüksek: veri + işlemler + senkronizasyon
Karşılıklı dışlama	Programcı wait/signal yerleştirir	Yapı tarafından otomatik sağlanır
Koşul bekleme	Sayma semaforlarıyla (ör. empty, full)	Koşul değişkenleriyle (wait / signal)
signal (bekleyen yokken)	Değeri artırır: “hatırlanır”	Etkisi yoktur: “kaybolur”
Programcı sorumluluğu	Yüksek	Daha düşük (ama sıfır değil)
Hata riski	Sıra hataları kolay	Daha yapılandırılmış; yine de kilitlenme / açlık olabilir
Tipik kullanım	Çekirdek, düşük düzey araçlar	Dil düzeyi yapılar, nesneye yönelik eşzamanlılık

Kaynak: Silberschatz vd., Bölüm 5.8

Monitör Semaforla Gerçekleştirilebilir mi? Evet



Monitör semafordan daha güçlü bir araç değildir; aynı şeyi daha güvenli bir arayüzle sunar. Ayrıntılı gerçekleştirme algoritması kitapta verilmiştir.

Kaynak: Silberschatz vd., Bölüm 5.8.3 (signal-and-wait için)

Gerçek Sistem Örneği: Pthreads Mutex ve Koşul Değişkeni

```
pthread_mutex_t m;   pthread_cond_t cv;

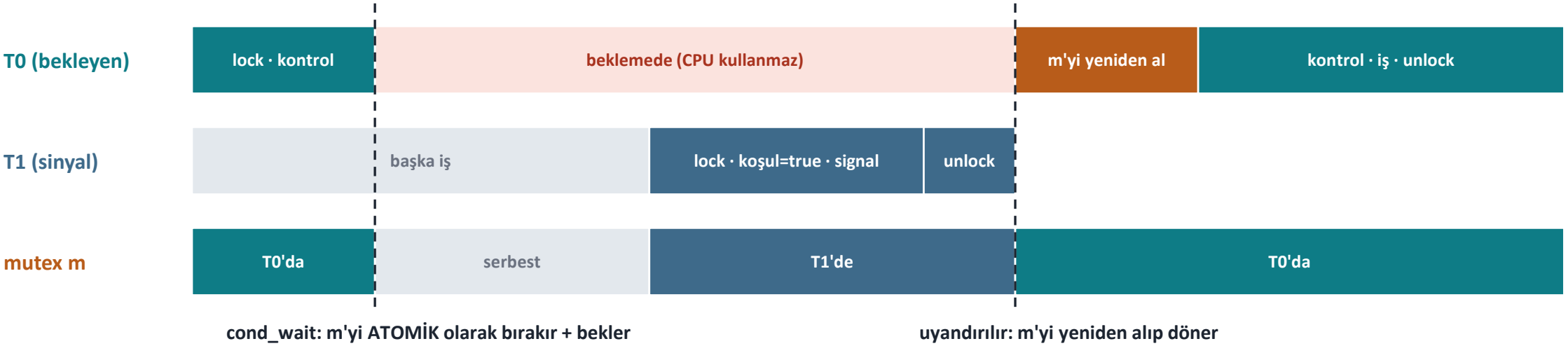
/* bekleyen taraf */
pthread_mutex_lock(&m);
while (!kosul)
    pthread_cond_wait(&cv, &m);
/* koşul doğru: işi yap */
pthread_mutex_unlock(&m);

/* koşulu değiştiren taraf */
pthread_mutex_lock(&m);
kosul = true;
pthread_cond_signal(&cv);
pthread_mutex_unlock(&m);
```

`pthread_mutex_t`	karşılıklı dışlama (monitör kilidinin karşılığı)
`pthread_cond_t`	koşul değişkeni: her zaman bir mutex ile birlikte kullanılır
`while (!kosul)`	koşul uyanınca yeniden sınanır
`pthread_cond_signal`	bekleyenlerden birini uyandırır

Kısa ve kavramsal örnek; hata denetimi ve ilk değer atama çıkarılmıştır. Kaynak: Silberschatz vd., Bölüm 5.9.4

pthread_cond_wait() Ne Yapar?



Neden atomik?

Bırakma ile bekleme arasına sinyal girerse sinyal kaybolabilir ve T0 sonsuza dek bekleyebilir.

Döndüğünde

T0 mutex'i tekrar tutuyordur; koşulu while ile yeniden sınar.

Kavramsal zaman çizelgesi

Kitap Temelli Sistem Örneği: Windows

KİTAP TEMELLİ SİSTEM ÖRNEĞİ

Çekirdek içinde

- Tek işlemcide: genel kaynağa erişirken ilgili kesmeler geçici olarak maskelenir
- Çok işlemcide: kısa kod bölümleri için döner kilitler; döner kilit tutan iş parçacığı kesilmez

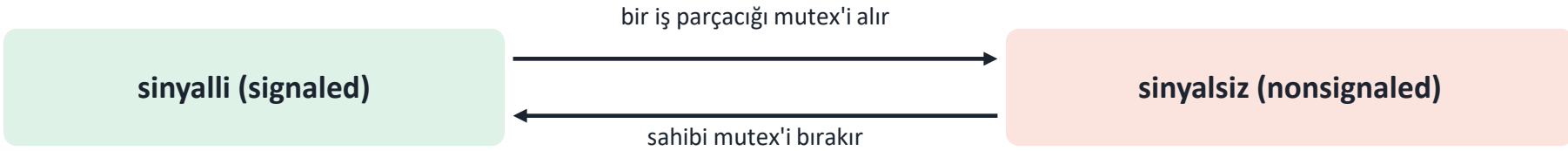
Çekirdek dışında: dağıtıcı nesneleri (dispatcher objects)

mutex: sahiplik ile korunan ortak veri

semafor: Bölüm 5.6'daki gibi

olay (event): koşul değişkenine benzer: bekleyene haber verir

zamanlayıcı (timer): belirli süre dolduğunu bildirir



Sinyalli nesne uygundur, alınırken bekletmez · sinyalsiz nesnede iş parçacığı bekler

Silberschatz vd., Bölüm 5.9.1 (Şekil 5.20) · Güncel Windows ayrıntıları eklenmemiştir

Kitap Temelli Sistem Örneği: Linux Çekirdeği

KİTAP TEMELLİ SİSTEM ÖRNEĞİ

Atomik tamsayılar (atomic_t)
Tüm matematik işlemleri kesintisiz; ör. paylaşılan bir sayacı kilitsiz güncellemek için verimli

Döner kilitler
Kısa süreli tutulan kilitler; çok işlemcide kullanılır

Mutex kilitleri ve semaforlar
Uzun süre tutulacak kilitler için: bekleyen uyur

Tek işlemcide
Döner kilit yerine çekirdek kesilmesi kapatılıp açılır (preempt_disable / enable)

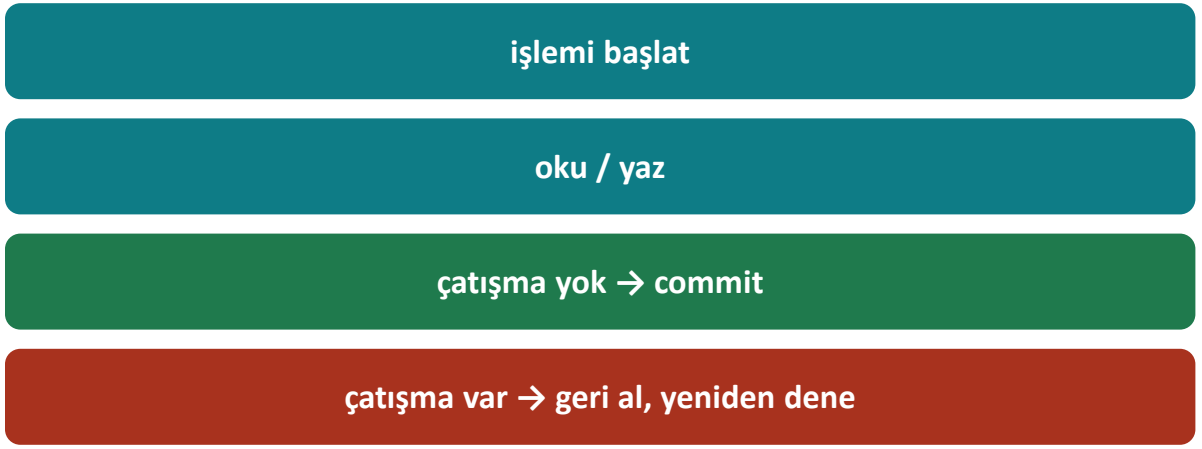
Kitapta belirtildiği gibi Linux çekirdeği 2.6 sürümüyle tamamen kesintili olmuştur; bu yüzden çekirdek verisi bu araçlarla korunur (Hafta 6: kesintili çekirdek).

Silberschatz vd., Bölüm 5.9.2 · Güncel çekirdek gerçekleştirmesi ayrıntılarına girilmemiştir

Alternatif Yaklaşım: İşlemsel Bellek (Transactional Memory)

Fikir: Bir grup okuma/yazma işlemi bir işlem (transaction) olarak yürütülür — veritabanlarından alınmış bir kavram.

```
void update() {
    atomic {
        /* paylaşılan veriyi değiştir */
    }
}
```



Amaç: açık kilit kullanımını azaltmak — kilitlenme olmaz, atomikliği sistem garanti eder. Yazılımsal (STM) ya da donanımsal (HTM) olarak gerçekleştirilebilir.

Kavramsal sözde kod. Kaynak: Silberschatz vd., Bölüm 5.10.1

Alternatif Yaklaşım: OpenMP ile Senkronizasyon

Hafta 4: #pragma omp parallel ile paralel bölge. Şimdi aynı çerçevenin senkronizasyon desteği:

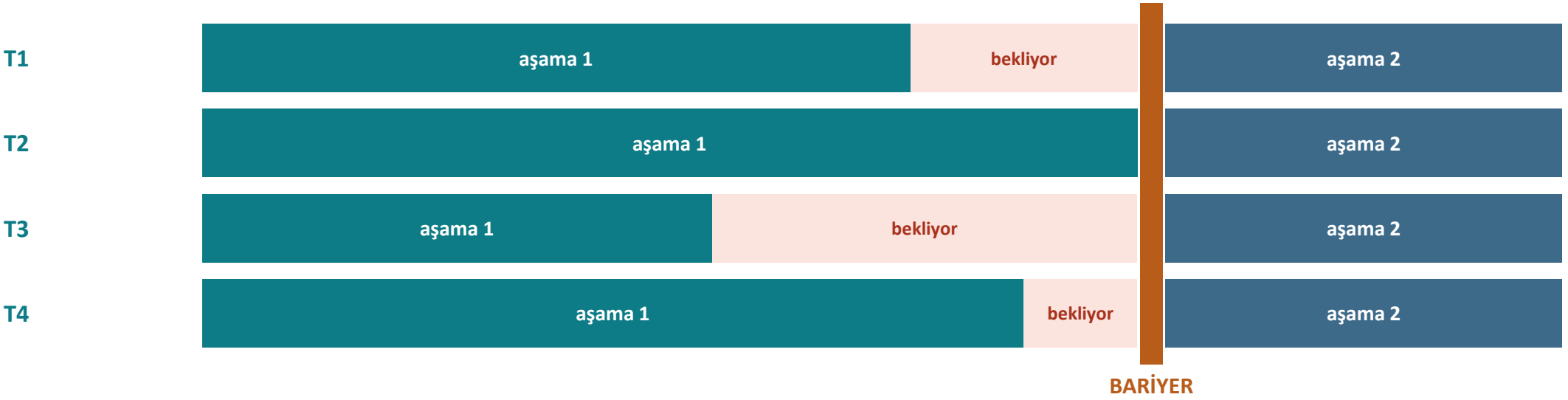
```
void update(int value) {
    #pragma omp critical
    {
        counter += value;
    }
}
```

- critical yönergesi altındaki bölge kritik bölgedir: aynı anda tek iş parçacığı yürütür
- Kilit yönetimini derleyici ve çalışma zamanı üstlenir
- Kullanımı kolaydır; ama kilitlenme gibi sorunları programcı yine de düşünmelidir

Ana fikir: Üst düzey çerçeveler senkronizasyonu dil yapısı olarak sunabilir.

Kaynak: Silberschatz vd., Bölüm 5.10.2

Bariyer (Barrier): Aşama Senkronizasyonu



Bir grup iş parçacığının hepsi bariyere ulaşmadan hiçbiri sonraki aşamaya geçemez. Son gelen (T2) ulaştığında hepsi birlikte devam eder.

Bariyer karşılıklı dışlama için bir kilit DEĞİLDİR: aynı anda birçok iş parçacığı çalışır; yalnızca aşama geçişini eşgüdümler.

Örnek kullanım: her aşaması bir önceki aşamanın tüm sonuçlarına ihtiyaç duyan paralel hesaplamalar. Kaynak: Tanenbaum & Bos, Bölüm 2.3.9

Kilitlerden Kaçınmak: Read-Copy-Update (RCU)

Çok okunan, az yazılan veri yapıları için (Tanenbaum & Bos, 2.3.10) — yalnızca kavramsal bakış

1. Okuyucular eski sürümü kilitsiz, çok düşük maliyetle okur
2. Yazar verinin YENİ bir kopyasını hazırlar ve değiştirir
3. Tek atomik işaretçi güncellemesiyle yeni sürüm yayımlanır: yeni okuyucular yeniyi görür
4. Eski sürümü okuyan kalmadığı güvenli zamanda eski kopya kaldırılır

Okuyucu her zaman ya tamamen eski ya tamamen yeni sürümü görür — asla yarım güncellenmiş veriyi değil.

Hangi Araç Ne Zaman?

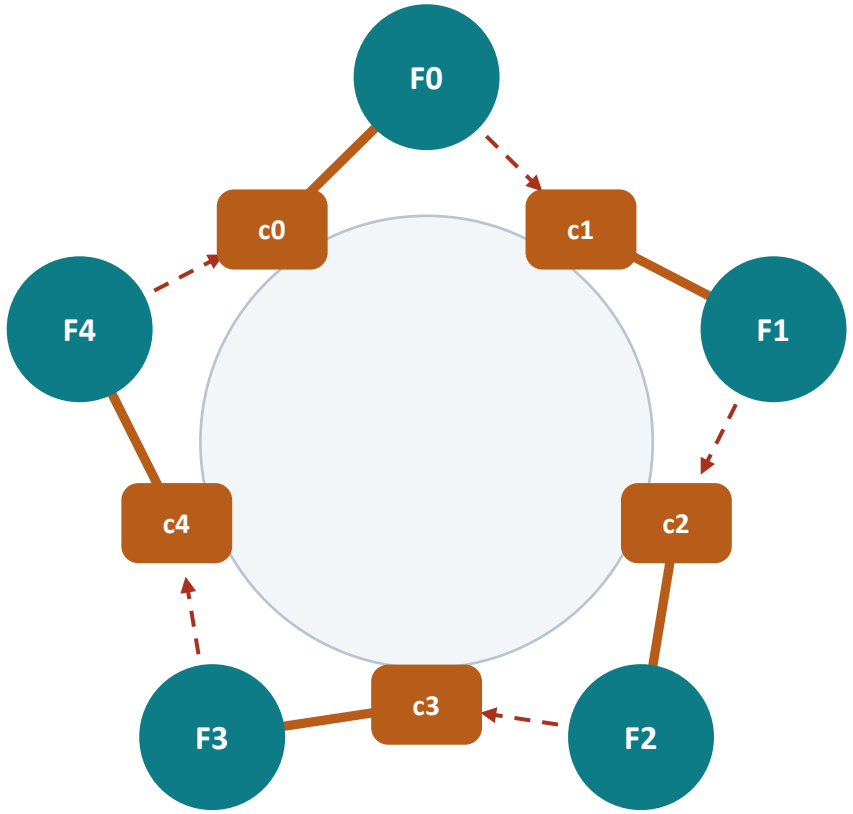
Araç	Karşılıklı dışlama	Kaynak sayma	Olay / koşul bekleme	Aşama senkronizasyonu	Soyutlama
Mutex	**evet**	hayır	hayır	hayır	düşük
Semafor	**evet (ikili)**	**evet (sayan)**	**evet (sinyalleşme)**	dolaylı	düşük
Koşul değişkeni	mutex/monitörle birlikte	hayır	**evet**	hayır	orta
Monitör	**otomatik**	koşullarla	**koşul değişkenleriyle**	hayır	yüksek
Bariyer	hayır	hayır	hayır	**evet**	orta

Sınırlı tampon: mutex + 2 sayan semafor (ya da monitör + 2 koşul)

Okuyucular–yazarlar: sayaç + 2 semafor; açlık politikası seçilmeli

Filozoflar: birden çok kaynak: kilitlenmeye dikkat

Bu Haftanın Ortak Gözlemi



Bu hafta iki kez gördük:

Sınırlı tampon: üretici mutex'i TUTUP empty BEKLİYOR; tüketici mutex'i BEKLİYOR

Filozoflar: her filozof bir çubuğu TUTUP diğerini BEKLİYOR

Soru: Her süreç bir kaynağı elinde tutup diğerini beklerse neden hiç kimse ilerleyemez? Bunu nasıl sistematik analiz eder, önler ya da tespit ederiz?

Hafta 8: Kilitlenmeler (Deadlocks)

Yaygın Yanlış Anlamalar

YANLIŞ	DOĞRU
“Üretici–tüketici için yalnızca mutex yeterli”	Tampon dolu/boş beklemesi için empty ve full da gerekir
“empty ve full karşılıklı dışlama sağlar”	Kaynak durumunu sayarlar; dışlamayı mutex sağlar
“Okuyucular hiçbir zaman aynı anda okuyamaz”	Birden çok okuyucu aynı anda okuyabilir
“Okuyucular–yazarlarda yazar açlığı olmaz”	Çözüm politikasına bağlı: okuyucu önceliğinde olabilir
“Filozofların naif çözümü kilitlenmez”	Herkes sol çubuğu alırsa kilitlenme oluşabilir
“Monitör = semafor”	Monitör, veri + işlemler + otomatik dışlamayı kapsülleyen üst düzey yapıdır
“Monitör her problemi otomatik çözer”	Kilitlenme ve açlık ayrıca değerlendirilmelidir
“condition.wait() monitörü tutarak bekler”	Bekler VE monitörü bırakır
“signal geldiyse koşul kesin doğrudur”	Koşul while ile yeniden sınanmalıdır
“Bariyer bir kilittir”	Aşama senkronizasyonudur; dışlama sağlamaz

Kontrol Soruları

SINIRLI TAMPON İZLEME

S1

N = 4; başlangıç empty = 4, full = 0, mutex = 1. Sırasıyla: Ü üretir, Ü üretir, T tüketir, Ü üretir, Ü üretir, Ü üretir, Ü üretir. Her adımdan sonra empty, full ve bekleyen süreci yazınız.

Hafta 6 kuralı: önce azalt; < 0 ise bekle.

S2

Son durumda bir tüketici bir öğe tüketirse empty ve full ne olur? Kim uyanır?

signal(empty) sırasında değer ≤ 0 mı?

Kontrol Soruları

YANLIŞ KOD ANALİZİ

S3

Üretici kodunda wait(mutex); wait(empty); sırası kullanılıyor. Hangi tampon durumunda bu sıra tehlikelidir? Olayları adım adım yazarak açıklayınız.

Kim neyi tutuyor, kim neyi bekliyor?

S4

Birinci okuyucular–yazarlar çözümünde read_count'u koruyan mutex kaldırılsaydı ne olabilirdi?

İki okuyucu aynı anda read_count++ yaparsa?

Kontrol Soruları

OKUYUCULAR–YAZARLAR · FİLOZOFLAR

S5

Sıra: R1 gelir, W1 gelir, R2 gelir, R1 çıkar, R2 çıkar. Her adımda read_count, rw_mutex değeri, okuyanlar ve bekleyen yazarı tablo hâlinde yazınız.

İlk okuyucu rw_mutex'i alır, son okuyucu bırakır.

S6

Naif semafor çözümünde beş filozofun kilitlenmesine yol açan bir yürütme sırası yazınız. Asimetrik sıra kuralı bu sırayı nasıl bozar?

Her filozof hangi çubuğu tutuyor, hangisini bekliyor?

Kontrol Soruları

MONİTÖR · KOŞUL DEĞİŞKENİ

S7 **condition.wait() çağrısı sırasında monitör kilidi (ya da pthread_cond_wait'te mutex) neden bırakılmak zorundadır? Bırakılmasaydı ne olurdu?**

İlgili bölüm: 4 · Monitörler

S8 **Monitörlü sınırlı tamponda remove() içinde while yerine if kullanılırsa, signal-and-continue semantiğinde hangi hata oluşabilir?**

Sinyal ile uyanan tüketicinin çalışması arasında ne olabilir?

DEĞERLENDİRME

Kontrol Soruları

KARŞILAŞTIRMA

S9 **Semafordaki signal() ile koşul değişkenindeki signal() arasındaki temel fark nedir? Bekleyen yokken her biri ne yapar?**

İlgili bölüm: 4 · Semafor ile monitör

S10 **Bariyer ile mutex arasındaki fark nedir? Hangi problem türü için hangisini kullanırsınız?**

İlgili bölüm: 5 · Alternatif yaklaşımlar

Kaynaklar

ANA KAYNAK

Silberschatz, A., Galvin, P. B., & Gagne, G. (2013). *Operating System Concepts* (9th ed.). John Wiley & Sons.

Bu hafta: Bölüm 5 — 5.7–5.10

DESTEKLEYİCİ KAYNAK

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.

Bu hafta: Bölüm 2 — 2.3.7, 2.3.9, 2.3.10 · 2.5.1–2.5.2

Not: Yürütme izleri Hafta 6'daki semafor kuralıyla (önce azalt; < 0 ise bekle; FIFO kuyruk) hazırlanmıştır. Windows ve Linux örnekleri kitap temellidir.